

Table of contents

Série 9 : Interprétation Abstraite I	1
Méthodes	1
Construction du CFG (Control Flow Graph)	1
Sémantique Collectrice (Collecting Semantics)	2
Résolution Itérative du Système d'Équations	2
Abstraction par Intervalles	2
Abstraction par Signes	2
Rappel	2
Exemples Illustratifs	3
Exemple 1 : Construction du CFG pour sillyprogram	3
Exemple 2 : Sémantique collectrice pour sillyprogram	4
Exemple 3 : Résolution concrète pour a=5, b=3	5
Exemple 4 : Abstraction par intervalles	5
Exemple 5 : Abstraction par signes	6
Synthèse	6

Série 9 : Interprétation Abstraite I

Méthodes

Construction du CFG (Control Flow Graph)

- **Méthode inductive** : Définir une fonction récursive $\text{CFG}(P, ip, rp)$ qui construit le graphe de flot de contrôle pour un programme P , avec un point d'entrée ip et un point de retour rp .
- **Règles de construction** :
 - **Affectation** : $\text{CFG}(x:=e, ip, rp)$ ajoute une transition étiquetée $x:=e$ de ip à rp .
 - **Séquence** : $\text{CFG}(e1; e2, ip, rp)$ crée un point intermédiaire $e1$ et appelle récursivement $\text{CFG}(e1, ip, e1)$ et $\text{CFG}(e2, e1, rp)$.
 - **Conditionnelle** : $\text{CFG}(\text{if } c \text{ then } e1 \text{ else } e2, ip, rp)$ crée deux nouveaux points $e1$ et $e2$, ajoute des transitions de ip à $e1$ (étiquette c) et de ip à $e2$ (étiquette $\neg c$), puis appelle récursivement $\text{CFG}(e1, e1, rp)$ et $\text{CFG}(e2, e2, rp)$.
 - **Boucle** : $\text{CFG}(\text{while } c \text{ do } e1 \text{ done}, ip, rp)$ crée un point $e1$, ajoute une transition de ip à $e1$ (étiquette c) et une de ip à rp (étiquette $\neg c$), puis appelle $\text{CFG}(e1, e1, ip)$.

Sémantique Collectrice (Collecting Semantics)

- **Système d'équations** : Associer à chaque point de contrôle s une variable $V(s)$ représentant l'ensemble des états de mémoire possibles en ce point.
- **Équations** : Dédire les équations à partir de la structure du CFG, en utilisant des opérations d'union et de filtrage par conditions.
- **Point d'entrée** : Introduire un état initial st pour modéliser l'initialisation des paramètres.

Résolution Itérative du Système d'Équations

- **Itération jusqu'à point fixe** : Initialiser $V(st)$ avec l'état initial, puis appliquer récursivement les équations jusqu'à stabilisation (ou divergence).
- **Analyse de terminaison** : Observer l'évolution des valeurs pour conclure sur le comportement du programme (terminaison, boucle infinie, erreur).

Abstraction par Intervalles

- **Domaine abstrait** : Remplacer les ensembles concrets de valeurs par des intervalles (ex: $[2, 6]$ pour $\{2, 3, 4, 5, 6\}$).
- **Opérations abstraites** : Adapter les opérations (affectation, union, intersection) pour travailler sur des intervalles.
- **But** : Simplifier l'analyse en perdant de la précision, mais en garantissant la terminaison et en capturant des propriétés générales.

Abstraction par Signes

- **Domaine abstrait** : Représenter les ensembles de valeurs par leur signe : 0 , 0 , <0 , >0 , $=0$, 0 , (aucune valeur), (toutes valeurs).
- **Opérations abstraites** : Définir les opérations sur les signes (ex: addition, soustraction, comparaison).
- **But** : Analyse encore plus grossière mais rapide, pour inférer des propriétés simples sur les variables.

Rappel

- **CFG** : Représentation graphique du flux de contrôle d'un programme. Les nœuds sont des points de contrôle, les arcs sont des transitions étiquetées par des conditions ou des affectations.

- **Sémantique collectrice** : Sémantique qui associe à chaque point de contrôle l'ensemble de tous les états de mémoire possibles lors de l'exécution.
- **Point fixe** : En interprétation abstraite, on cherche un point fixe du système d'équations (un état où $V(s)$ ne change plus après une itération). L'algorithme itératif converge vers ce point fixe (sous certaines conditions).
- **Abstraction** : Fonction qui map un ensemble concret (ex: $\{2,3,4\}$) vers un élément d'un domaine abstrait (ex: intervalle $[2,4]$ ou signe >0). Elle doit être sûre (sur-approximative).
- **Opérations abstraites** : Doivent sur-approximer les opérations concrètes. Par exemple, l'addition d'intervalles $[a,b] + [c,d] = [a+c, b+d]$.

Exemples Illustratifs

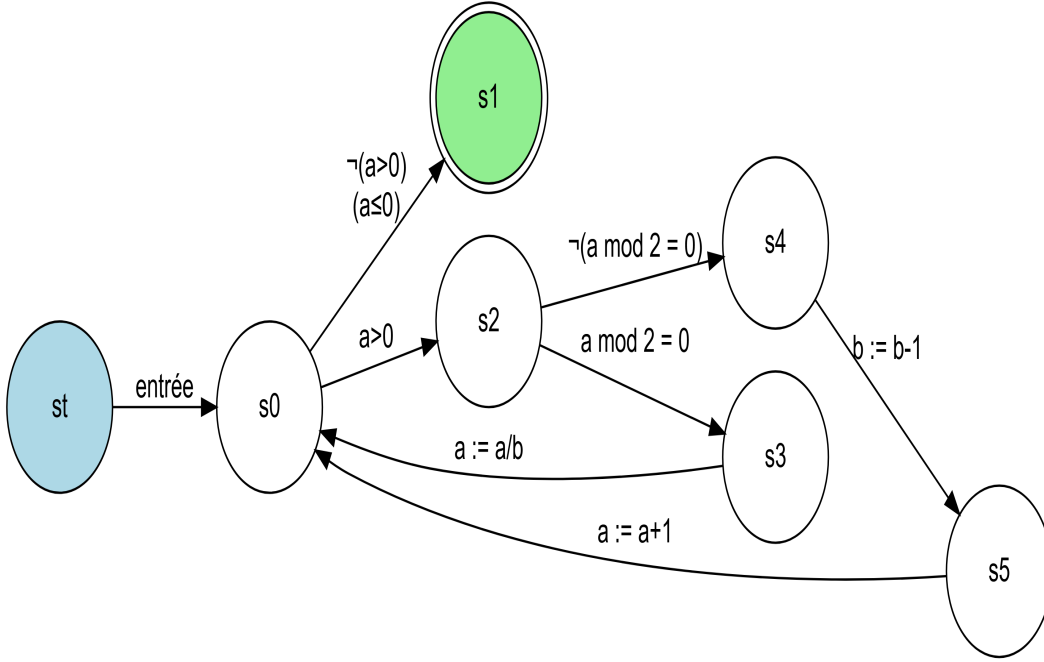
Exemple 1 : Construction du CFG pour sillyprogram

Contexte : Programme `sillyprogram(a,b) = while (a > 0) do if a mod 2 = 0 then a := a/b else b := b-1; a := a+1 endif done`. On veut construire son CFG.

Application : On applique les règles inductives.

1. **Boucle while** : $CFG(\text{while } (a>0) \text{ do } \dots \text{ done, ip, rp})$
 - Crée un nouveau point $e1$ pour le corps de la boucle.
 - Ajoute une transition de ip à $e1$ étiquetée $a>0$.
 - Ajoute une transition de ip à rp étiquetée $\neg(a>0)$ (c'est-à-dire $a\leq 0$).
 - Appel récursif : $CFG(\text{corps, } e1, ip)$ où `corps` est l'instruction `if ... endif`.
2. **Conditionnelle if** : $CFG(\text{if } a \bmod 2 = 0 \text{ then } a:=a/b \text{ else } b:=b-1; a:=a+1, e1, ip)$
 - Crée deux points $e2$ (pour le then) et $e3$ (pour le else).
 - Ajoute une transition de $e1$ à $e2$ étiquetée $a \bmod 2 = 0$.
 - Ajoute une transition de $e1$ à $e3$ étiquetée $\neg(a \bmod 2 = 0)$.
 - Appels récursifs :
 - $CFG(a:=a/b, e2, ip)$
 - $CFG(b:=b-1; a:=a+1, e3, ip)$
3. **Séquence** : $CFG(b:=b-1; a:=a+1, e3, ip)$
 - Crée un point intermédiaire $e4$.
 - Appels récursifs :
 - $CFG(b:=b-1, e3, e4)$
 - $CFG(a:=a+1, e4, ip)$

Le CFG résultant a des points de contrôle **st** (initial), **s0** (début de la boucle), **s1** (sortie de la boucle), **s2** (début du if), **s3** (branche then), **s4** (branche else), **s5** (après **b:=b-1**). Les transitions sont étiquetées comme décrit.



Exemple 2 : Sémantique collectrice pour sillyprogram

Contexte : On a le CFG de **sillyprogram**. On veut écrire le système d'équations de la sémantique collectrice.

Application : On définit $V(s)$ pour chaque point de contrôle **s**.

$$\begin{aligned}
 V(st) &= \top \quad (\text{état initial non spécifié}) \\
 V(s0) &= V(st)[a := a_0, b := b_0] \cup V(s3)[a := a/b] \cup V(s5)[a := a + 1] \\
 V(s1) &= V(s0) \cap [a \leq 0] \\
 V(s2) &= V(s0) \cap [a > 0] \\
 V(s3) &= V(s2) \cap [a \bmod 2 = 0] \\
 V(s4) &= V(s2) \cap [a \bmod 2 \neq 0] \\
 V(s5) &= V(s4)[b := b - 1]
 \end{aligned}$$

Ici, $[\text{cond}]$ dénote l'ensemble des états satisfaisant la condition **cond**. L'opération $V(s)[x := \text{expr}]$ signifie "prendre tous les états de $V(s)$, évaluer **expr** dans chaque état, et assigner la valeur résultante à **x**".

Exemple 3 : Résolution concrète pour $a=5$, $b=3$

Contexte : On résout le système d'équations pour les valeurs initiales $a=5$, $b=3$.

Application : On itère manuellement.

- Initialisation : $V(s_0) = \{a:\{5\}, b:\{3\}\}$
- Première itération (boucle) :
 - Par s_2 (car $a>0$) : $V(s_2) = \{a:\{5\}, b:\{3\}\}$
 - Par s_4 (car $a \bmod 2 = 0$) : $V(s_4) = \{a:\{5\}, b:\{3\}\}$
 - Par s_5 : $V(s_5) = \{a:\{5\}, b:\{2\}\}$
 - Retour à s_0 : $V(s_0) = \{a:\{5\}, b:\{3\}\}$ $\{a:\{6\}, b:\{2\}\} = \{a:\{5,6\}, b:\{2,3\}\}$
- Deuxième itération : On recalcule avec le nouveau $V(s_0)$. On obtient $V(s_0) = \{a:\{2,3,5,6\}, b:\{1,2,3\}\}$.
- Troisième itération : $V(s_0) = \{a:\{0,1,2,3,5,6\}, b:\{0,1,2,3\}\}$.
- Quatrième itération : $V(s_0) = \{a:\{0,1,2,3,5,6\}, b:\{-1,0,1,2,3\}\}$.
- Cinquième itération : Les ensembles continuent de s'élargir (apparition de valeurs négatives). On conclut que la boucle ne termine pas et que les valeurs divergent.

Exemple 4 : Abstraction par intervalles

Contexte : On reprend le même système, mais on abstrait les ensembles de valeurs par des intervalles.

Application : On ré-exprime les équations avec des intervalles.

- Initialisation : $V(s_0) = \{a:[5,5], b:[3,3]\}$
- Première itération :
 - $V(s_2) = \{a:[5,5], b:[3,3]\}$
 - $V(s_4) = \{a:[5,5], b:[3,3]\}$
 - $V(s_5) = \{a:[5,5], b:[2,2]\}$
 - $V(s_0) = \{a:[5,5], b:[3,3]\}$ $\{a:[6,6], b:[2,2]\} = \{a:[5,6], b:[2,3]\}$
- Deuxième itération : On obtient $V(s_0) = \{a:[2,6], b:[1,3]\}$.
- Troisième itération : $V(s_0) = \{a:[0,6], b:[0,3]\}$.
- Quatrième itération : $V(s_0) = \{a:[0,6], b:[-1,3]\}$.
- Cinquième itération : $V(s_0) = \{a:[-6,6], b:[-2,3]\}$ (en approximant les valeurs négatives apparues). L'intervalle pour a s'élargit vers les négatifs, et on peut conjecturer qu'il deviendra $(-\infty, +\infty)$ après suffisamment d'itérations.

Exemple 5 : Abstraction par signes

Contexte : On utilise maintenant l'abstraction par signes.

Application : On traduit les valeurs en signes.

- Initialisation : $V(s_0) = \{a:>0, b:>0\}$ (car $5>0, 3>0$)
- Première itération :
 - $V(s_2) = \{a:>0, b:>0\}$
 - $V(s_4) = \{a:>0, b:>0\}$
 - $V(s_5) = \{a:>0, b:0\}$ (car $b-1$ peut être $2>0$, mais on sur-approche par 0)
 - Retour à s_0 : Union des signes : $\{a:>0, b:>0\} \cup \{a:>0, b:0\} = \{a:>0, b:\}$ (car $>0 \cup 0 =$ pour b)
- Deuxième itération : Avec $b:$, les calculs abstraits donneront des signes très larges. Rapidement, on obtiendra $V(s_0) = \{a:, b:\}$, c'est-à-dire que toute valeur est possible pour a et b . Cela indique que l'analyse par signes est trop grossière pour ce programme, mais elle permet de conclure que le programme peut ne pas terminer (car on ne peut pas garantir que a deviendra 0).

Synthèse

L'interprétation abstraite est une technique d'analyse statique qui permet d'approximer le comportement d'un programme sans l'exécuter concrètement. On construit d'abord un CFG, puis on définit un système d'équations de sémantique collectrice. La résolution itérative de ce système peut ne pas terminer sur des programmes complexes. Pour garantir la terminaison, on utilise des domaines abstraits (intervalles, signes) qui simplifient les données et permettent de calculer un point fixe en un nombre fini d'étapes. Cependant, l'abstraction entraîne une perte de précision : on peut obtenir des résultats sûrs mais parfois trop larges.